Chapter 6 - Assets layout and web performance

Practical source and control reference
29 October 2026

Use this guide to find a feature, understand its owner, make a bounded change, and check how that change is saved. Paths are relative to C:/Dropbox/Resume Website unless stated otherwise.

Contents

Keep authoring originals and web exports distinct 2

Trace an asset from source to screen 2

Improve a material without making the model heavy 2

Replace a model safely 2

Layout and hit areas 3

Practical performance checks 3

Folder migration gate 3

Keep authoring originals and web exports distinct

Original Blender/design files belong with their authoring sources; GLB, compressed images/audio and manifests are runtime products. Preserve the original and the recipe that produces the runtime copy. A smaller exported file can legitimately omit editor-only content, but its required node names and material/UV conventions remain contracts.

Assets can live under shared `assets/` or inside a feature directory. Prefer shared placement when several owners use the same resource, and local placement when one feature owns it. Repeated named `images` folders are easier to navigate than a single unclassified pile.

Trace an asset from source to screen

1. Find the runtime URL in the page/module, manifest, CSS or HTML.
2. Follow `core/assets.js` and `asset-policy.js` to determine local versus hosted resolution.
3. Find the editable original and exporter/import recipe. Check whether a manifest, atlas, thumbnail or low-resolution proxy must be regenerated too.
4. Preserve names, case, dimensions and stable IDs unless updating every consumer/migration deliberately.
5. Check the artifact policy in `tools/release-artifact.json`; being present on disk does not mean the file is packaged.
6. Verify a direct local request and actual decoding/rendering. After an authorized release, check the public object separately.

Current hosted families include `Videos/`, `Renders/`, `music/`, `midi/`, `soundfonts/` and recognized model paths. `assets/glb/` maps to hosted `glb/`. Do not move these directories solely to make Explorer look cleaner without preserving external keys and callers.

Improve a material without making the model heavy

The desk provides a worked example. Its original wood/leather Mapping Scale was supplied by a Value node. The glTF exporter dropped that linked constant, causing excessive texture repetition and flat-looking mip averages. Increasing image size alone would waste bandwidth without fixing that error.

`tools/export-docs-desk.py` now writes the same mapping constants into the export-time material sockets. Original source files remain untouched. Key color maps use at most 1024px; normal/scalar/hardware maps stay at 512px. The model remains 23 meshes and 23,461 exported vertices, with a roughly 3.4 MB self-contained GLB.

For another material fault, compare UV set/index, UV transform, color space, normal convention, roughness/metallic packing and supported shader nodes before adding geometry or huge textures. Procedural Blender shading may need baking, but only bake the channels actually missing. Keep normal/roughness data out of sRGB interpretation.

Inspect an imported web GLB rather than only the source blend. Good source rendering does not prove that the runtime export contains the intended material data.

Replace a model safely

1. Record its named nodes, parent hierarchy, transforms, bounds, materials and any saved anchors/animation targets.

2. Export to a separate candidate first. Preserve the old asset until the candidate's contracts are checked.
3. Keep the intended world scale and orientation. Replacing a normalized desk with raw source coordinates would also move papers and break framing.
4. Check input targets, child hardware, source-local offsets and bounding boxes. Drawer travel is a parent-local vector, not an arbitrary world translation.
5. Check animation, suspension, contrast/reduced motion and disposal, then inspect from several angles.
6. Replace the reviewed runtime asset and preserve a specific rollback copy. Do not save the source just to export it.

Saved sticky-note anchors can depend on named surfaces and triangle/plane conventions. Geometry replacement may need explicit anchor migration; do not silently attach old notes to unrelated triangles.

Layout and hit areas

Native page layout generally belongs to page CSS/HTML and shared shell styles. Three.js layout can be authored in a model, computed by a feature module, or projected from a camera. Identify which layer owns the coordinate before editing it.

For a button, change both its visual location and picking target. For a 3D panel, use feature-local coordinates so it moves with the object. For Settings/Debug centering, use the shared `panel-viewport.js` measurement rather than hardcoded drawer widths. Preserve the user's pan/rotation while accommodating UI space.

Do not independently offset WebGL and CSS3D layers that must overlap. Check picking after resize, panel movement, model rotation, focus zoom and device-pixel-ratio changes.

Practical performance checks

- Network: compare exported bytes and number of requests. Compression reduces transfer, not necessarily decoded GPU memory.
- Textures: record actual dimensions, channel formats, unique image count and mip overhead. Keep small UI hardware maps small.
- Geometry: record meshes/primitives and vertices/triangles; avoid subdivision merely to carry grain that a normal map can show.
- Animation: update only required state, reuse cached textures/materials and avoid adding a second persistent RAF loop for a small control.
- Lifetime: dispose owned textures/materials/geometry once and remove listeners/timers. Do not dispose borrowed cube resources.
- Measurement: compare the same view/device/content and cache state. Local diagnostic-server numbers and headless renders are not production device performance.

Folder migration gate

File reorganization is deferred pending owner coordination and human approval. Plan old/new paths, dependent imports/URLs, generators, public keys, saved-data migration and rollback first. Legacy patch utilities can rewrite current HTML/CSS or depend on their own location; inspect them before running or moving them.